Cyberscope

A TAC Security Company

Audit Report

VaultedMetrics Token

March 2026

Network: ETH

Address: 0x062570121e9519C77CB71C7936B6C1f76C7B1119

0x88C8b2AEaA1c0701B97E23dF36d53D5c1722B49a

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	VM888Token VM888Vesting
Compiler Version	v0.8.20+commit.a1b79de6
Optimization	200 runs
Explorer	https://etherscan.io/address/0x062570121e9519C77CB71C7936B6C1f76C7B1119#code https://etherscan.io/address/0x88C8b2AEaA1c0701B97E23dF36d53D5c1722B49a#code
Address	0x062570121e9519C77CB71C7936B6C1f76C7B1119
	0x88C8b2AEaA1c0701B97E23dF36d53D5c1722B49a
Network	ETHEREUM
Symbol	VM888
Decimals	18
Total Supply	888,000,000

Audit Updates

Initial Audit	16 Feb 2026 https://github.com/cyberscope-io/audits/blob/main/vm888/v1/audit.pdf
Corrected Phase 2	03 Mar 2026

Source Files

Filename	SHA256
VM888VestingCyberScope.sol	f03bff63f9d62d25fe3c8fc78118f05480c91 03ce01fedd7df443e36e5308a25
VM888Cyberscope.sol	94cf3b18b57c206d119e30642b7d5fcc07 a1bf19d47753eff3b628638d073f33

Overview

VM888Cyberscope.sol

Vaulted Metrics Token is an ERC-20 token with integrated Reflect Finance (RFI) mechanics, where non-excluded holders passively earn reflections through a shrinking reflected supply `(_rTotal)`. The fixed supply of 888,000,000 tokens is minted entirely to the deployer at construction. Every transfer routes through `_transfer()`, which enforces pause state, blacklist checks, trading activation, per-transaction and per-wallet caps, and a 30-minute sell cooldown. Transfers are classified as buys, sells, or wallet-to-wallet, each subject to distinct tax schedules comprising reflections, auto-LP, treasury, and staking components, with additional dynamic loyalty and whale dump surcharges on sells. An anti-snipe mechanism applies a 99% tax during the first blocks after trading is enabled.

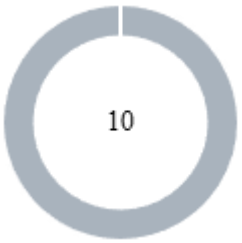
Tax tokens accumulate in the contract and are periodically swapped to ETH via `_swapAndDistribute()`, which adds permanent liquidity (LP tokens burned to `0xdead`), distributes ETH to treasury and staking wallets, and funds a buyback reserve. An autonomous buyback engine monitors the spot price against a lazily-updated 30-day EMA; when price falls below EMA, the contract buys and burns tokens. The contract is owner-controlled with administrative functions for blacklisting, exclusions, pause control, and token recovery. `renounceOwnership()` automatically unpauses and permanently disables all admin functions.

VM888VestingCyberScope.sol

Vaulted Metrics Vesting implements a linear vesting schedule for three hardcoded team allocations totaling 66,600,000 VM888: Founder (70%), VP (20%), and a Future Reserve (10%) whose address is set post-deployment. The contract must be pre-funded before `startVesting()` initializes the vesting clock. Vesting accrues linearly over 5 years from `vestingStart`, with a 12-month cliff preventing any claims during the first year. The `claim()` function transfers the entire claimable balance in a single transaction — partial claims are not supported. Before executing, `claim()` verifies three conditions: the cliff has elapsed, unclaimed vested tokens exist, and the current spot price (read from the token contract) equals or exceeds 120% of the 30-day EMA, preventing team sells during

downtrends. The contract uses a custom ownership model with `renounceOwnership()` for full autonomy, and an emergency `recoverERC20()` that explicitly excludes VM888 to protect beneficiary allocations.

Findings Breakdown



Critical	0
Medium	0
Minor / Informative	10

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	0	10	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	AME	Address Manipulation Exploit	Acknowledged
●	IAC	Inaccurate APY Calculation	Acknowledged
●	IBP	Inaccurate Buffer Progression	Acknowledged
●	MV	Misleading Variables	Acknowledged
●	PLPI	Potential Liquidity Provision Inadequacy	Acknowledged
●	PMRM	Potential Mocked Router Manipulation	Acknowledged
●	PTAI	Potential Transfer Amount Inconsistency	Acknowledged
●	PTRP	Potential Transfer Revert Propagation	Acknowledged
●	SCB	Sell Cooldown Bypass	Acknowledged
●	UPR	Unprotected Price Reading	Acknowledged

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L352
Status	Acknowledged

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations

Shell

```
function recoverERC20(address tokenAddress,
uint256 amount)
external onlyOwner {
require(tokenAddress != address(vm888), "Cannot
recover
VM888");
(bool success, bytes memory data) =
tokenAddress.call(
abi.encodeWithSelector(0xa9059cbb, msg.sender,

amount)
);
require(success && (data.length == 0 ||
abi.decode(data, (bool))), "Transfer failed");
}
```

Recommendation

The team is advised to ensure that the `getAPY()` function returns accurate values that reflect the actual 7-day reflection history at the time of the query, regardless of transaction activity or the time elapsed since the last transfer.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

IAC - Inaccurate APY Calculation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L1001
Status	Acknowledged

Description

The contract maintains a 7-day circular buffer to track daily reflections for APY estimation. The buffer is updated exclusively during token transfers, as the `_checkDayBoundary()` function only executes when piggybacked on a transaction. The `getAPY()` function reads the buffer directly without accounting for the time elapsed since the last update. As a result, during periods of inactivity, the function may return values that do not accurately represent the current 7-day reflection history.

```
Shell
function getAPY() external view returns (uint256)
{
    uint256 totalReflections7Days;
    for (uint256 i = 0; i < APY_WINDOW; i++) {
        totalReflections7Days +=
dailyReflections[i];
    }
    uint256 avgDailyReflections =
totalReflections7Days / APY_WINDOW;
    ...
    return (avgDailyReflections * 365 * 10000) /
tCirculating;}
```

Recommendation

The team is advised to ensure that the `getAPY()` function returns accurate values that reflect the actual 7-day reflection history at the time of the query, regardless of transaction activity or the time elapsed since the last transfer.

IBP - Inaccurate Buffer Progression

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L978
Status	Acknowledged

Description

The contract maintains a 7-day circular buffer to track daily reflections for APY estimation. The buffer is updated by the `_checkDayBoundary()` function, which is called within `_executeTransferWithTax()`. However, the reflection fee is accumulated into `todayReflections` before `_checkDayBoundary()` is called. As a result, when the first transaction of a new day triggers the boundary check, the reflections from that transaction are included in the previously activated buffer index rather than the position that corresponds to the current day. This causes a misallocation where the current day's reflections are recorded in the wrong buffer position, effectively allocating it in an earlier day. Subsequent iterations of the circular buffer, will result to the effect of that day being overwritten earlier than expected, leading to misleading APY values

Shell

```
function _checkDayBoundary() private {
 if (block.timestamp >= lastDayRecorded + 1 days) {
    uint256 daysElapsed = (block.timestamp -
lastDayRecorded) / 1 days;

    dailyReflections[currentDayIndex] = todayReflections;
    currentDayIndex = (currentDayIndex + 1) % APY_WINDOW;

    uint256 daysToClear = daysElapsed > APY_WINDOW ?
APY_WINDOW : daysElapsed;
    for (uint256 i = 1; i < daysToClear; i++) {
        dailyReflections[currentDayIndex] = 0;
        currentDayIndex = (currentDayIndex + 1) %
APY_WINDOW;
    }

      todayReflections = 0;
      lastDayRecorded = block.timestamp;
    }
}

function getAPY() external view returns (uint256) {
 uint256 totalReflections7Days;
 for (uint256 i = 0; i < APY_WINDOW; i++) {
    totalReflections7Days += dailyReflections[i];
 }
 uint256 avgDailyReflections = totalReflections7Days /
APY_WINDOW;
    ...
 return (avgDailyReflections * 365 * 10000) /
tCirculating;

}
```

Recommendation

The team is advised to ensure that the clearing logic remains consistent and that running values are allocated to the correct indices. This will maintain the accuracy of the reported APY regardless of transaction frequency.

MV - Misleading Variables

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L174,182,189,800
Status	Acknowledged

Description

Variables can have misleading names if their names do not accurately reflect the value they contain or the purpose they serve. The contract uses some variable names that are too generic or do not clearly convey the information stored in the variable. Misleading variable names can lead to confusion, making the code more difficult to read and understand.

```
Shell
address public constant STAKING_WALLET =
0x40bA7dB618BD9eE95E9745D598629c0fF69c84eE;

uint256 public constant BUY_STAKING_BPS = 300;

uint256 public constant SELL_STAKING_BPS = 300;
...

(bool stakingSuccess,) =
payable(STAKING_WALLET).call{value: ethForStaking}("");
```

Recommendation

It's always a good practice for the contract to contain variable names that are specific and descriptive. The team is advised to keep in mind the readability of the code.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

PLPI - Potential Liquidity Provision Inadequacy

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L820,837,862
Status	Acknowledged

Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

```
Shell
function _swapTokensForETH(uint256 tokenAmount) private {
    ...

uniswapRouter.swapExactTokensForETHSupportingFeeOnTransferT
okens(

function _addLiquidity(uint256 tokenAmount, uint256
ethAmount) private {
    ...
    uniswapRouter.addLiquidityETH{value: ethAmount}(


uniswapRouter.swapExactETHForTokensSupportingFeeOnTransferT
okens{value: ethToSpend}
```

Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

PMRM - Potential Mocked Router Manipulation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L312,318
Status	Acknowledged

Description

The contract accepts the router address as an argument during deployment. While this feature provides flexibility, it introduces a security threat. The owner could set the router address to any contract that implements the router's interface, potentially containing malicious code. Since `uniswapRouter` is declared as immutable, this address cannot be corrected after deployment. In the event of a transaction triggering the swap functionality with such a malicious contract as the router, all internal operations including auto-swaps, liquidity additions, and buybacks may be manipulated.

```
Shell
constructor(address _router) Ownable(msg.sender) {
    ...
    IUniswapV2Router02 router =
IUniswapV2Router02(_router);

    uniswapRouter = router;
```

Recommendation

The team should consider including the router address as a constant immutable value in the codebase.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L231
Status	Acknowledged

Description

The `claim` function updates the `totalClaimed` state variable with the expected transfer amount. However, the VM888 token implements a fee mechanism on transfers. If the vesting contract is not excluded from tax, the actual amount received by the beneficiary could be less than the recorded amount. As a result, there will be an inconsistency between the tracked claimed amount and the actual amount received by the beneficiary.

```
Shell
totalClaimed[msg.sender] += amount;

if (vm888.balanceOf(address(this)) < amount) revert
InsufficientContractBalance();


bool success = vm888.transfer(msg.sender, amount);
```

Recommendation

The team is advised to ensure that the vesting contract address is excluded from the token's tax mechanism. This will guarantee that the full expected amount is received by the beneficiary during claims, maintaining consistency between the tracked and the actual transferred amount.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

PTRP - Potential Transfer Revert Propagation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L793,800
Status	Acknowledged

Description

The contract sends funds to `TREASURY_WALLET` and `STAKING_WALLET` as part of the transfer flow. These addresses can either be wallet addresses or contracts. If any of the addresses belongs to a contract then it may revert from incoming payment. As a result, the error will propagate to the token's contract and revert the transfer.

```
Shell
(bool treasurySuccess,) =
payable(TREASURY_WALLET).call{value: ethForTreasury}("");

(bool stakingSuccess,) =
payable(STAKING_WALLET).call{value: ethForStaking}("");
```

Recommendation

The contract should tolerate the potential revert from the underlying contracts when the interaction is part of the main transfer flow. This could be achieved by not allowing set contract addresses or by sending the funds in a non-revertable way.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

SCB - Sell Cooldown Bypass

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L216,431
Status	Acknowledged

Description

The contract implements a time-delayed sell restriction using a 30-minute cooldown period to limit repeated sell executions. However, the cooldown accounting is tied to the seller's address rather than the tokens themselves. As a result, a user can distribute tokens across multiple wallets and execute sell transactions from each wallet independently without ever triggering the cooldown restriction, effectively bypassing the intended rate limiting guarantees.

```shell
uint256 public constant SELL_COOLDOWN = 30 minutes;


function _transfer(address from, address to, uint256
amount)
...
 if (isSell && !isExcludedFromTax[from]) {
            if (block.timestamp <
lastSellTimestamp[from] + SELL_COOLDOWN) {
                revert SellCooldownActive();
            }
            lastSellTimestamp[from] = block.timestamp;

        }
```

Recommendation

The team is advised to scope cooldown tracking to a mechanism that cannot be trivially circumvented by cycling wallet addresses. This will enforce cooldown semantics at the token level, preserve the safety assumptions behind sell rate limiting, and ensure sell operations cannot be replayed in rapid succession by distributing tokens across multiple addresses.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

UPR - Unprotected Price Reading

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L861,909,929, VM888VestingCyberScope.sol#L220
Status	Acknowledged

Description

The contract determines whether a buyback should be triggered and how the EMA price reference should update by relying on spot price quotations derived from the Uniswap V2 pair reserves at the moment of the transaction. These quotations are obtained via `_getCurrentPriceRaw()` which reflects the current state of the pool, not a time-averaged or manipulation-resistant price. As a result, the EMA update and buyback trigger are directly coupled to transient reserve conditions that can change within the same block. An attacker can trade against the VM888/WETH pool to temporarily distort the reserve ratio, causing the EMA to record a manipulated price or triggering an unnecessary buyback. The attacker can then sandwich the resulting buyback swap to extract value from the ETH reserve. Compared to a standard sandwich attack, the impact is amplified because the manipulated price not only worsens execution but also directly affects the contract's internal EMA tracking and autonomous buyback decisions.

```shell
function _getCurrentPriceRaw() private view returns
(uint256)
     ...
     return (reserveETH * 1e18) / reserveToken;

function _updateEMA() private {
    uint256 currentPrice = _getCurrentPriceRaw();
        ...
 currentEMA = (EMA_ALPHA * currentPrice +
EMA_ONE_MINUS_ALPHA * currentEMA) / 1e18;
```

```
function _tryBuyback() private {
    ...
    if (currentPrice >= currentEMA) return;
```

The same unprotected price reading is also used by the vesting contract's `claim()` function to verify that the current price exceeds 120% of the EMA before allowing beneficiaries to withdraw their vested tokens. Since this check relies on the same spot price quotation, a beneficiary can temporarily inflate the pool price within a single transaction to satisfy the price condition, bypassing the only protection that prevents team members from claiming tokens during market downtrends

Shell

```
uint256 currentPrice = vm888.getCurrentPrice();
uint256 ema = vm888.getEMA();

if (ema > 0) {
    uint256 requiredPrice = (ema *
PRICE_THRESHOLD_MULTIPLIER) / 1e18;

    if (currentPrice < requiredPrice) revert
PriceConditionNotMet();
```

Recommendation

Price-dependent protocol decisions such as EMA updates and buyback triggers should not rely on instantaneous spot prices derived from liquidity pool reserves. The team is advised to incorporate a price reading mechanism that does not depend on mutable pool state at the moment of transfer

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

.

Functions Analysis

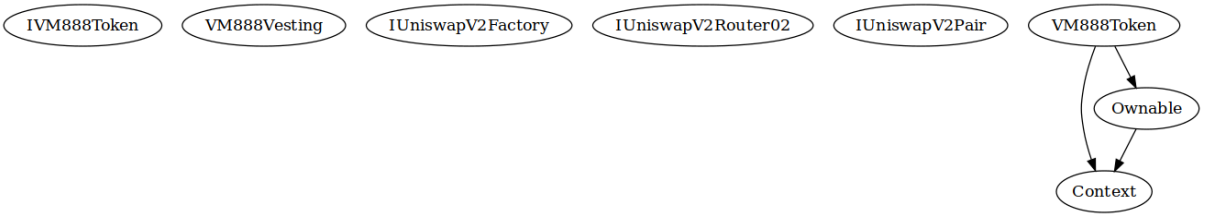
Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
IVM888Token	Interface			
	transfer	External	✓	-
	balanceOf	External		-
	getCurrentPrice	External		-
	getEMA	External		-
VM888Vesting	Implementation			
		Public	✓	-
	startVesting	External	✓	onlyOwner
	vestedAmount	Public		-
	claimable	Public		-
	claim	External	✓	onlyBeneficiary
	getAllocation	External		-
	remainingLocked	External		-
	getPriceCondition	External		-
	getVestingStatus	External		-
	timeUntilCliff	External		-
	vestingPercentComplete	External		-
	setFutureReserveAddress	External	✓	onlyOwner

	transferOwnership	External	✓	onlyOwner
	renounceOwnership	External	✓	onlyOwner
	recoverERC20	External	✓	onlyOwner
	_getAllocation	Private		
	_isBeneficiary	Private		
IUniswapV2Factory	Interface			
	createPair	External	✓	-
	getPair	External		-
IUniswapV2Router02	Interface			
	factory	External		-
	WETH	External		-
	addLiquidityETH	External	Payable	-
	swapExactTokensForETHSupportingFeeOnTransferTokens	External	✓	-
	swapExactETHForTokensSupportingFeeOnTransferTokens	External	Payable	-
IUniswapV2Pair	Interface			
	getReserves	External		-
	token0	External		-
	totalSupply	External		-
Context	Implementation			

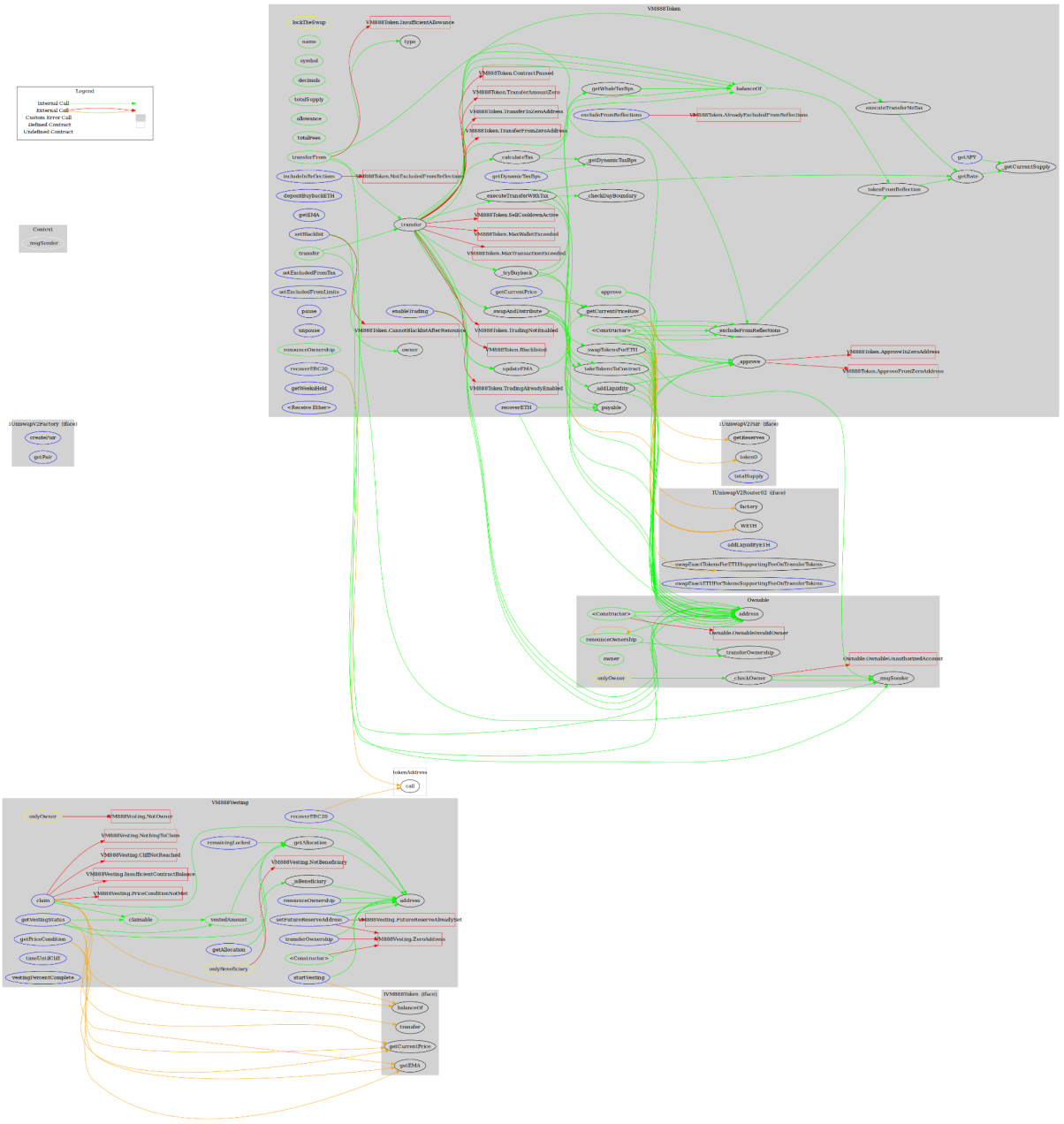
	_msgSender	Internal		
Ownable	Implementation	Context		
		Public	✓	-
	owner	Public		-
	_checkOwner	Internal		
	_transferOwnership	Internal	✓	
	renounceOwnership	Public	✓	onlyOwner
VM888Token	Implementation	Context, Ownable		
		Public	✓	Ownable
	name	Public		-
	symbol	Public		-
	decimals	Public		-
	totalSupply	Public		-
	balanceOf	Public		-
	allowance	Public		-
	totalFees	Public		-
	approve	Public	✓	-
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	_approve	Private	✓	
	_transfer	Private	✓	
	_calculateTax	Private		

	_getDynamicTaxBps	Private		
	_getWhaleTaxBps	Private		

Inheritance Graph



Flow Graph


VM888Token
lockTheSwap
name
type
symbol
decimals
totalSupply
allowance
totalFees
transferFrom
includeInReflections
depositBuybackETH
getEMA
setBlacklist
transfer
setExcludedFromTax
setExcludedFromLimits
pause
unpause
renounceOwnership
recoverERC20
getWeeksHeld
<Receive Ether>
VM888Token.InsufficientAllowance
VM888Token.ContractPaused
VM888Token.TransferAmountZero
VM888Token.TransferInZeroAddress
VM888Token.TransferFromZeroAddress
VM888Token.NotExcludedFromReflections
transfer
VM888Token.SellCooldownActive
VM888Token.MaxWalletExceeded
VM888Token.MaxTransactionExceeded
_tryBuyback
getCurrentPrice
swapAndDistribute
VM888Token.CannotBlackListAfterRenounce
enableTrading
owner
VM888Token.TradingNotEnabled
VM888Token.BlackListed
updateEMA
VM888Token.TradingAlreadyEnabled
recoverETH
getWhaleTaxBps
balanceOf
excludeFromReflections
VM888Token.AlreadyExcludedFromReflections
executeTransferNoTax
calculateTax
getDynamicTaxBps
executeTransferWithTax
checkDayBoundary
tokenFromReflection
getAPY
getRate
getCurrentSupply
approve
getCurrentPriceRaw
<Constructor>
swapTokensForETH
takeTokensToContract
addLiquidity
payable
VM888Token.ApproveToZeroAddress
VM888Token.ApproveFromZeroAddress
IUniswapV2Factory (iface)
createPair
getPair
IUniswapV2Pair (iface)
getReserves
token0
totalSupply
IUniswapV2Router02 (iface)
factory
WETH
addLiquidityETH
swapExactTokensForETHSupportingFeeOnTransferTokens
swapExactETHForTokensSupportingFeeOnTransferTokens
Ownable
address
Ownable.OwnableInvalidOwner
transferOwnership
checkOwner
Ownable.OwnableUnauthorizedAccount
_msgSender
tokenAddress
call
VM888Vesting
onlyOwner
VM888Vesting.NotOwner
VM888Vesting.NothingToClaim
VM888Vesting.CliffNotReached
VM888Vesting.InsufficientContractBalance
VM888Vesting.PriceConditionNotMet
claim
getVestingStatus
getPriceCondition
timeUntilCliff
vestingPercentComplete
claimable
vestedAmount
getAllocation
remainingLocked
recoverERC20
getAllocation
VM888Vesting.NotBeneficiary
_isBeneficiary
renounceOwnership
address
setFutureReservesAddress
transferOwnership
<Constructor>
startVesting
onlyBeneficiary
VM888Vesting_FutureReservesAlreadySet
VM888Vesting.ZeroAddress
IVM888Token (iface)
balanceOf
transfer
getCurrentPrice
getEMA
Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
Context
_msgSender

Summary

VaultedMetrics contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. The Smart Contract analysis reported no compiler error or critical issues.

The ownership of the VM888Token token has been renounced on the following transaction:

https://etherscan.io/tx/0x992bd56a9c91697b2f95d83d89e9588e9bb28ae53bb75c7f6240292e86a024b0

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io